

A low-angle, upward-looking photograph of several modern skyscrapers with glass and steel facades, reaching towards a clear blue sky. The perspective creates a sense of height and architectural grandeur.

Topic: *Using Online Analytical Processing*

Lesson 4 - 5: Week 4 - 5 Laboratory

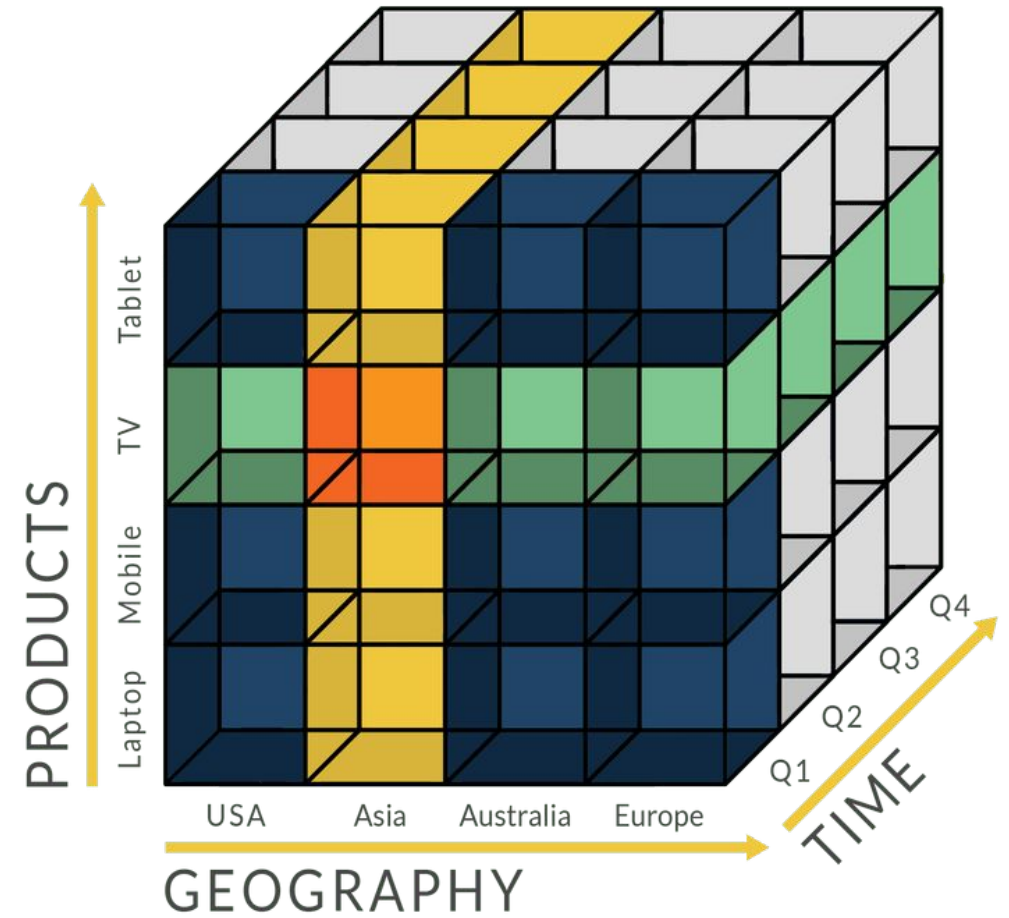
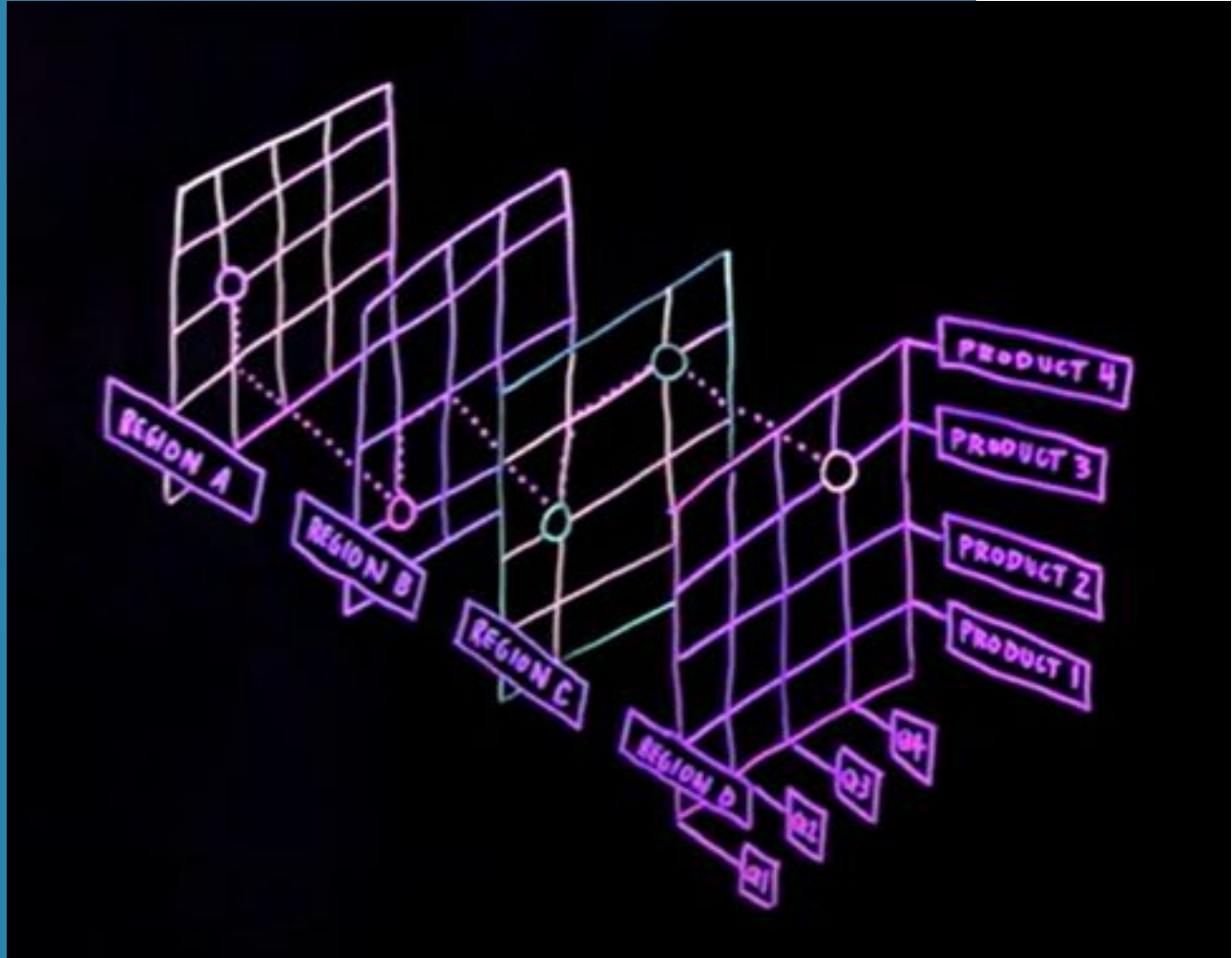
Introduction



Presented by Jared HILLAM, BI Practice Director

OLAP AND OTHER OPTIONS

- OLAP (online analytical processing) is a computing method that enables users to easily and selectively extract and query data in order to analyze it from different points of view. OLAP business intelligence queries often aid in trends analysis, financial reporting, sales forecasting, budgeting and other planning purposes.

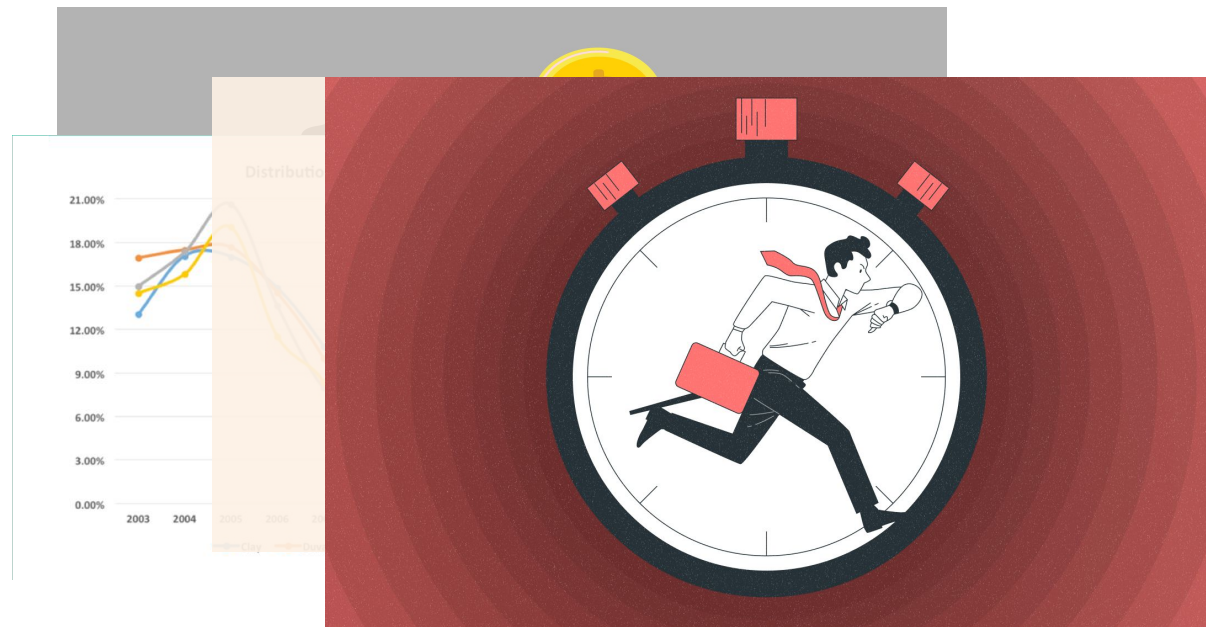


Online Analytical Processing (OLAP) is a process of data analysis for information stored in a relational database. It is primarily involved in aggregating large amounts of diverse data. It is considered as the widely used information analysis process in most business intelligence solutions today. With OLAP, a database administrator or database expert can perform data analysis by finding data relationships and look for patterns, trends and exceptions. The result is then used by the company in making strategic and tactical decisions.

To further understand OLAP, it is better to understand first a few terms. OLAP is based on facts and all facts are numbers. The numbers in the OLAP spreadsheet are called facts.

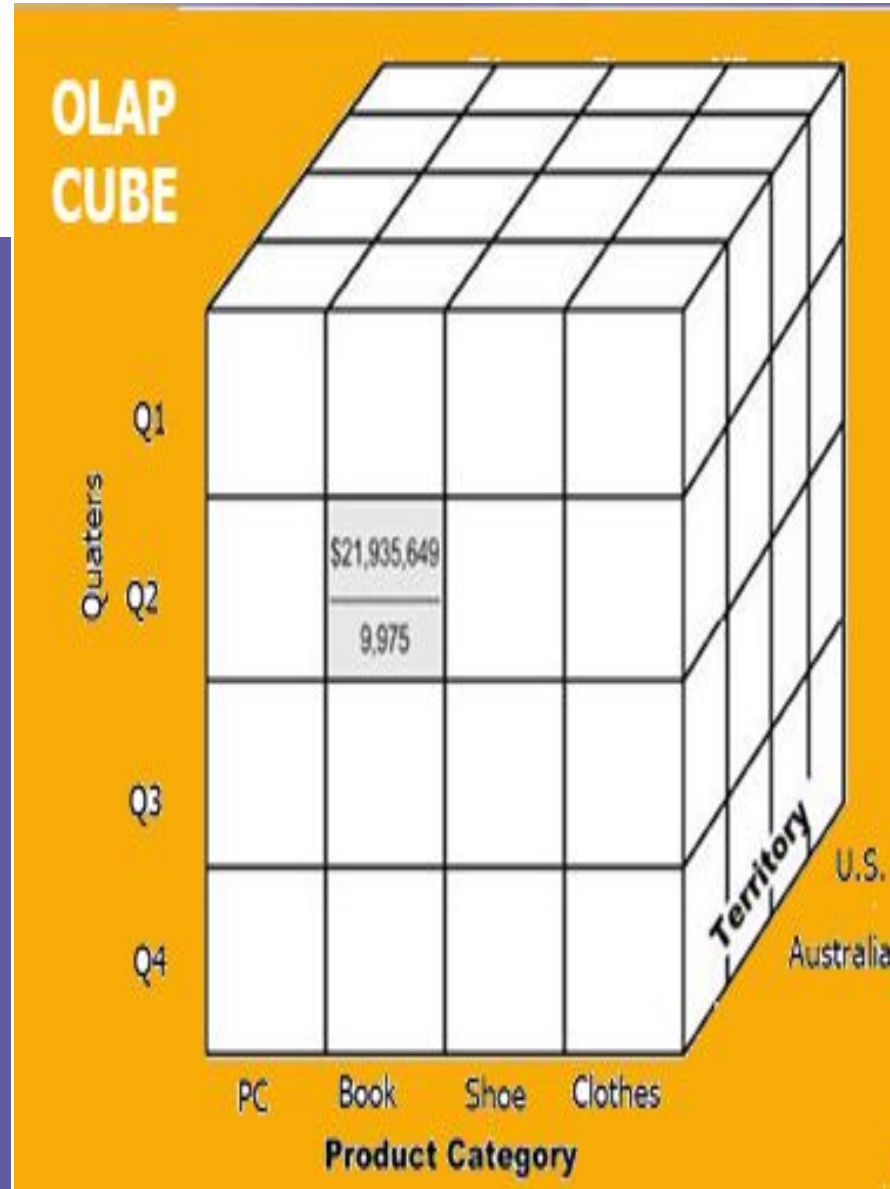
Typical facts would be:

- Sales Quantity
- Sales Count
- Profit
- Hours of Work



OLAP

- A fact can be a count of sales, sum of the order amounts or an average of sales revenue. These facts are then organized into dimensions. The dimensions are the ways the facts can be broken down. For example, total orders might be able to be broken up by geography. Moreover, total orders might also be broken down by time. Dimensions can also be hierarchies of levels. For example, a geography dimension might contain the levels of country and state. In addition, in a time dimension, it can be broken down by year, month, and day. Another important term is cube. Cubes are collections of facts and dimensions. For example, a cube can be used to represent production data. Similarly, a separate cube can be used to represent sales data.

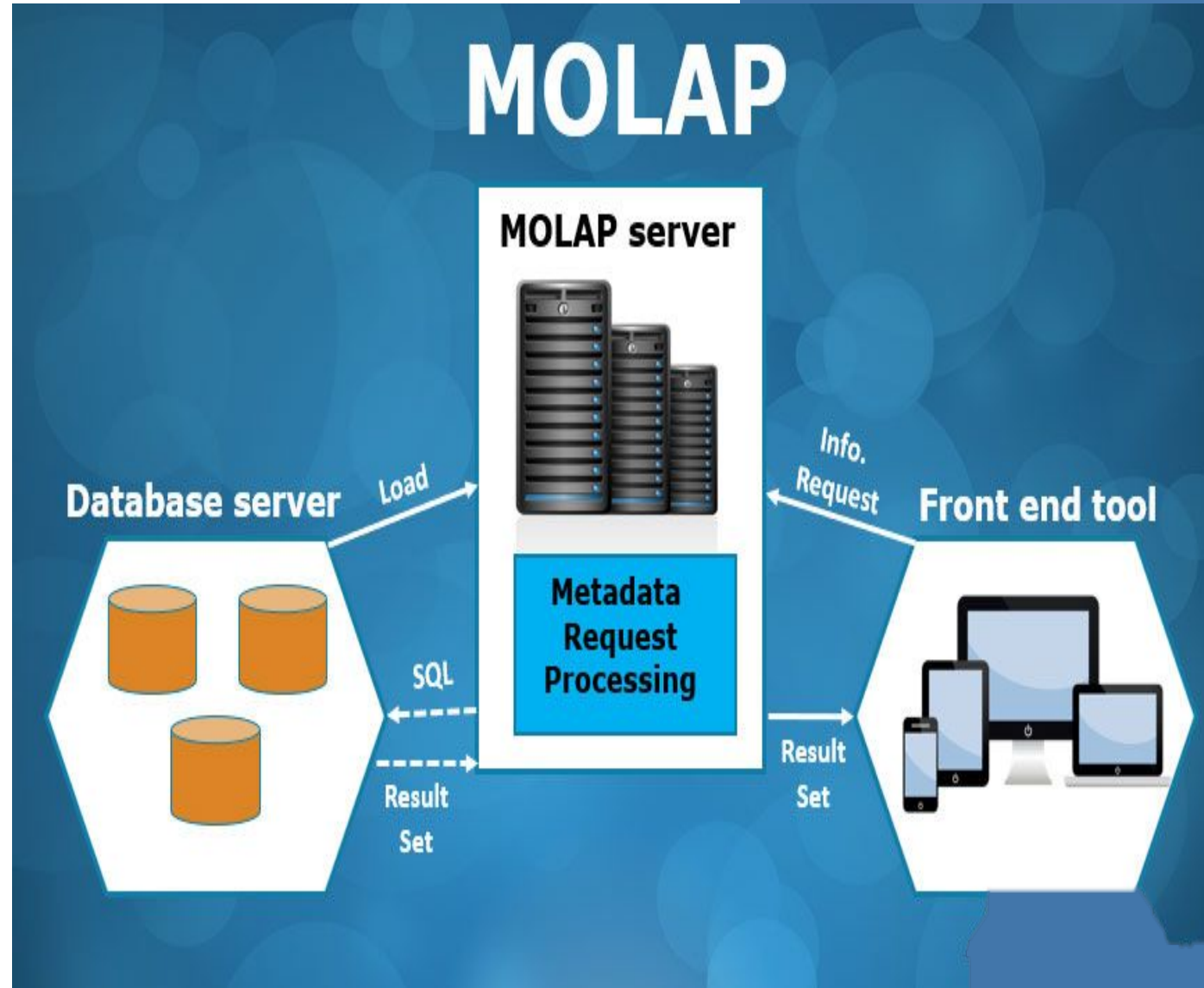


The OLAP functionality for some RDBMS has a separate OLAP module. The advantage of having a separate module is that it is portable and customization is easier. The disadvantage is that data has to be moved from the database into the OLAP module before it is analyzed. Some database products such as Adaptive Server Anywhere have OLAP feature built into the database itself. The advantage of this is it makes deployment and integration with other database features easier and more seamless. For these reasons, the shift toward using the OLAP functionality within the RDBMS is a future direction.

Types of OLAP

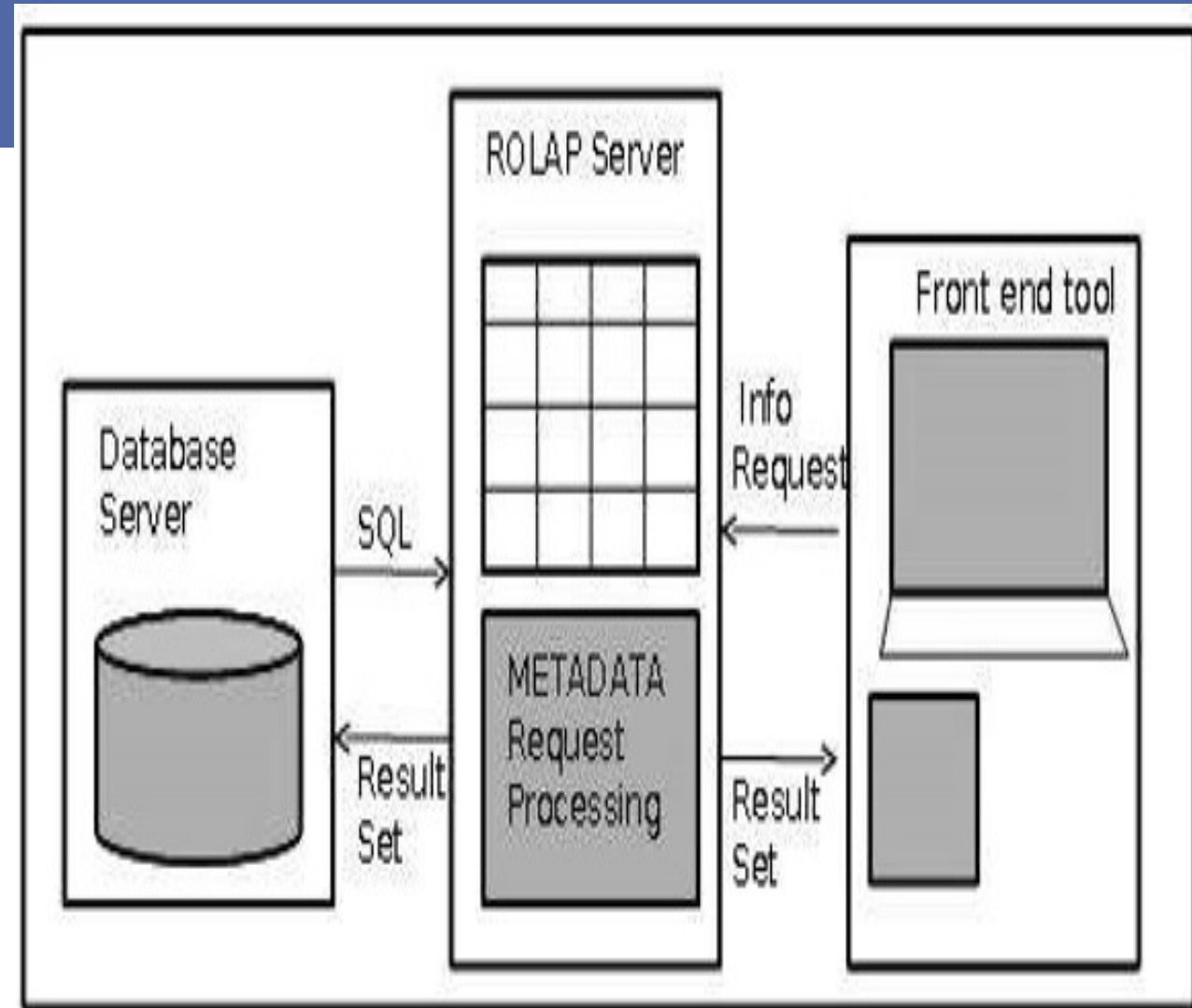
There are three prominent architectures for OLAP systems namely MOLAP, ROLAP, and HOLAP.

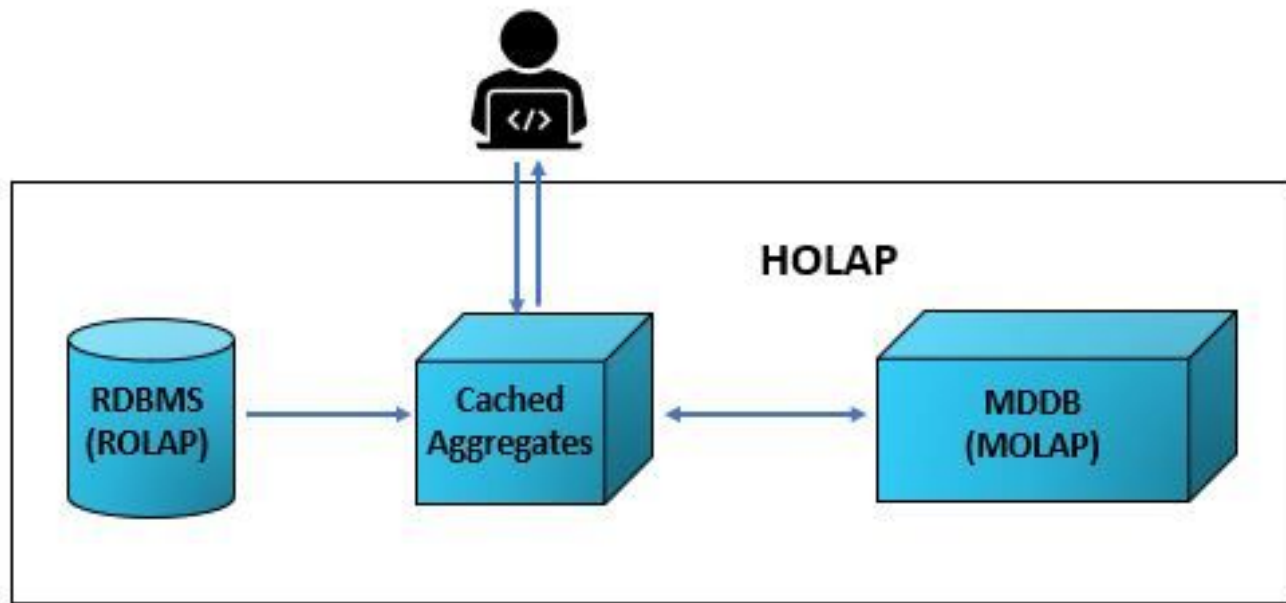
Multidimensional Online Analytical Processing (MOLAP) - is considered as the most standard approach to OLAP solutions. It directly stores the information contained in the various cubes using a multidimensional database. It is best described by storing data multi-dimensionally. This architecture is commonly used in multidimensional databases.



Relational Online Analytical Processing (ROLAP)

Relational Online Analytical Processing (ROLAP) - uses the same solution as MOLAP but uses a relational database for storage of the data. This solution translates native OLAP queries which are written in a language called multidimensional expressions (MDX) into the appropriate SQL statements. This is to prevent the need for another copy of the data. It uses the data created directly by the online transaction processing (OLTP) application or from data warehouses. A good example for this architecture would be an ORACLE table.

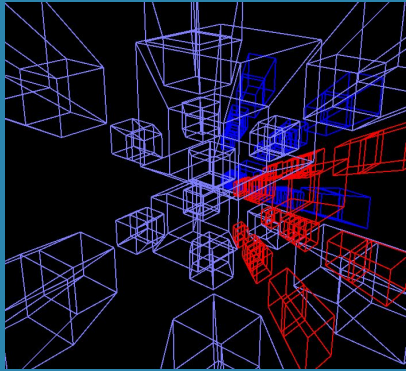




Hybrid Online Analytical Processing (HOLAP)

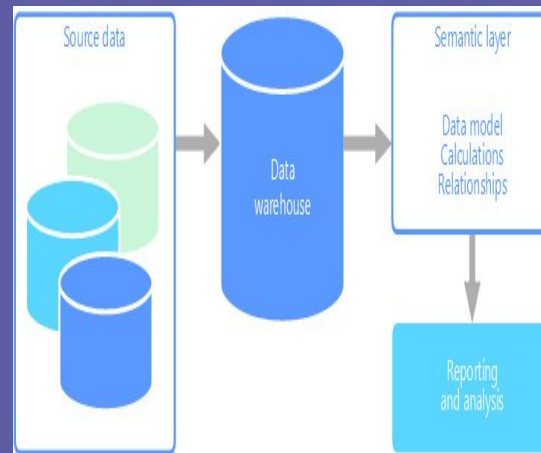
Hybrid Online Analytical Processing (HOLAP) - is a hybrid approach that combines MOLAP and ROLAP. First, the aggregated totals are stored in a multidimensional database then the detail data is stored in the relational database. This solution implements a balance between the data efficiency of the ROLAP model and the performance of the MOLAP model. This architecture offers access to diverse data sources on local and remote servers.

Advantages of Using OLAP



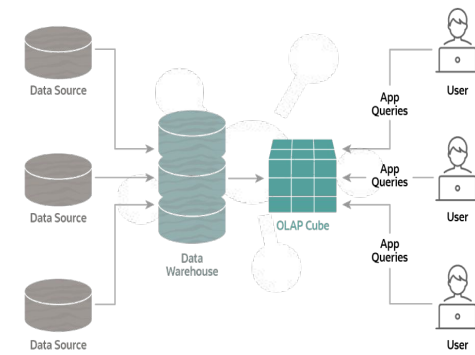
- Multidimensional - allows operational managers to analyze numerical values from different perspectives, e.g. product, time, and geography.

- Consistently fast - OLAP pre-aggregate data to ensure fast and predictable query times.



The OLAP Process

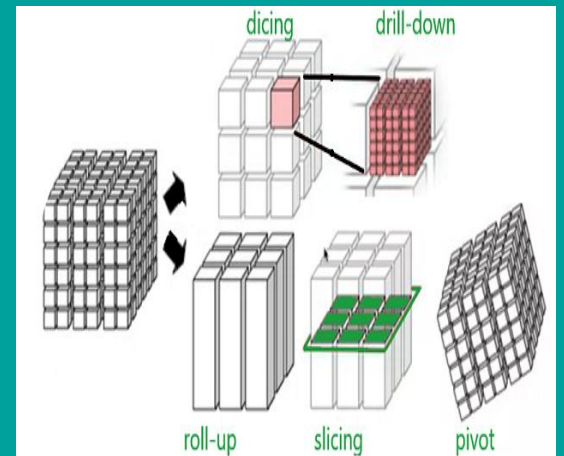
How data is prepared for online analytical processing (OLAP)



OLAP works by extracting data from multiple sources and formatting it into cubes, which can then be analyzed from multiple points of view. Multiple cubes can be nested, creating multidimensional "hypercubes".

- Intuitive interface - quick manipulation and analysis of data and generation of reports

- Complex calculations - with multiple dimensions comes more complex, cross-dimensional calculations.



Using Rollup Operation

The ROLLUP operation is used when you want to get a result set showing totals and subtotals. It adds subtotal rows into the result sets of queries with GROUP BY clauses. ROLLUP generates a result set showing aggregates for a hierarchy of values.

The rollup operator is similar to the group by operator except it gives you subtotals and grand totals at the end. The syntax is:

```
SELECT ...  
FROM ...  
WHERE ...  
GROUP BY ROLLUP (column1,  
column2,...)
```

- From the syntax, the DBMS will perform a group by on the columns listed as well as provide a subtotal every time a new grouping is created for the set of columns and finally a grand total for all rows in the result set. Here are some examples in using the ROLLUP operation:

Example 1

```
SELECT year (order_date) AS Year, quarter (
order_date) AS
Quarter, COUNT (*) AS Orders
FROM sales_order
GROUP BY ROLLUP (Year, Quarter)
ORDER BY Year, Quarter
```

The above query returns data that summarizes the number of sales orders by year and quarter. Table 4.1 represents the result set from the query. The subtotal rows are highlighted in the result set. Each subtotal row has a NULL value in the column or columns over which the subtotal is computed.

	Year	Quarter	Orders
1	(NULL)	(NULL)	648
2	2000	(NULL)	380
3	2000	1	87
4	2000	2	77
5	2000	3	91
6	2000	4	125
7	2001	(NULL)	268
8	2001	1	139
9	2001	2	119
10	2001	3	10

Example 2

	Year	Quarter	Region	Orders
1	(NULL)	(NULL)	(NULL)	92
2	2000	(NULL)	(NULL)	43
3	2000	1	(NULL)	23
4	2000	1	South	12
5	2000	1	Western	11
6	2000	2	(NULL)	20
7	2000	2	South	12
8	2000	2	Western	8
9	2001	(NULL)	(NULL)	49
10	2001	1	(NULL)	30
11	2001	1	South	14
12	2001	1	Western	16
13	2001	2	(NULL)	19
14	2001	2	South	11
15	2001	2	Western	8

The second example produces a complicated result set. The result set summarizes the number of sales orders by year, quarter, and region. In this example, only the first and second quarters and two selected regions (South and the Western region) are examined.

```
SELECT year (order_date) AS Year, quarter
(order_date) AS
Quarter, region, COUNT (*) AS Orders
FROM sales_order
WHERE region IN ('South', 'Western') AND quarter IN
('1', '2')
GROUP BY ROLLUP (Year, Quarter, Region)
ORDER BY Year, Quarter, Region
```

Using Cube Operation

The CUBE operation is useful when you want to work with a large amount of data in your reports. Like ROLLUP, the CUBE operator provides subtotals of aggregate values in the result set. It produces subtotals for all possible combinations of the columns listed in the group by clause. When the CUBE operation is performed on variables, the result set includes many subtotal rows based on combinations of the values of the variables.

The CUBE operator returns a result set with added information of dimensions to the data. You can further analyze columns of your data, which are referred to as dimensions. The result set contains all possible combinations of the values in the dimension columns, together with the aggregate values from the underlying rows that match the combination of dimension values. CUBE provides a cross tabulation report of all possible combinations of the dimensions and generates a result set that shows aggregates for all combinations of values in selected columns.

The syntax is:

```
SELECT ...  
  
FROM ...  
  
WHERE ...  
  
GROUP BY CUBE (column1, column2, ..
```

Here is an example in using the CUBE operation:

The example query below returns a result set that summarizes the total number of orders and then calculates subtotals for the number of orders by year and quarter.

```
SELECT year (order_date) AS Year,  
quarter  
(order_date) AS  
Quarter, COUNT (*) AS Orders  
FROM sales_order  
GROUP BY CUBE (Year, Quarter)  
ORDER BY Year, Quarter
```

	Year	Quarter	Orders
1	(NULL)	(NULL)	648
2	(NULL)	1	226
3	(NULL)	2	196
4	(NULL)	3	101
5	(NULL)	4	125
6	2000	(NULL)	380
7	2000	1	87
8	2000	2	77
9	2000	3	91
10	2000	4	125
11	2001	(NULL)	268
12	2001	1	139
13	2001	2	119
14	2001	3	10

Using Grouping Sets Operation

GROUPING SETS allows you to compute groups on several different sets of grouping columns in the same query. It lets you to explicitly specify which subtotals to add as compared to CUBE and ROLLUP which add a predefined set of subtotals into the result set. The GROUPING SETS operation is useful if you would like to return only a part of the multidimensional result set of a cube. It permits you to select which variables to compare instead of returning a comparison of all the variables like CUBE, or a hierarchical subset like ROLLUP.

The SQL syntax is:

```
SELECT ...  
  
FROM ...  
  
WHERE ...  
  
GROUP BY GROUPING SETS ((column1,  
column2),  
  
(column2), ...)
```

Using Grouping Sets Operation

	Year	Quarter	Orders
1	2000	(NULL)	380
2	2000	1	87
3	2000	2	77
4	2000	3	91
5	2000	4	125
6	2001	(NULL)	268
7	2001	1	139
8	2001	2	119
9	2001	3	10

Here is an example in using the **GROUPING SETS** operation:

```
SELECT year (order_date) AS Year,  
quarter
```

```
(order_date) AS
```

```
Quarter, COUNT (*) AS Orders
```

```
FROM sales_order
```

```
GROUP BY GROUPING SETS ((Year,  
Quarter), (Year))
```

```
ORDER BY Year, Quarter
```

The query specifies that the result set returns subtotal rows for each year, but not for the individual quarters. The table below represents the result set from the query.

OLAP Functions

OLAP functions provide the capability to perform analytic tasks on data, such as computing moving averages, ranks, and cumulative values. You can include an OLAP function in a select-list or on the ORDER BY clause of a SELECT statement. An OLAP function cannot be used as an argument of an aggregate function. Therefore, you cannot have functions like `SUM( RANK() )`.

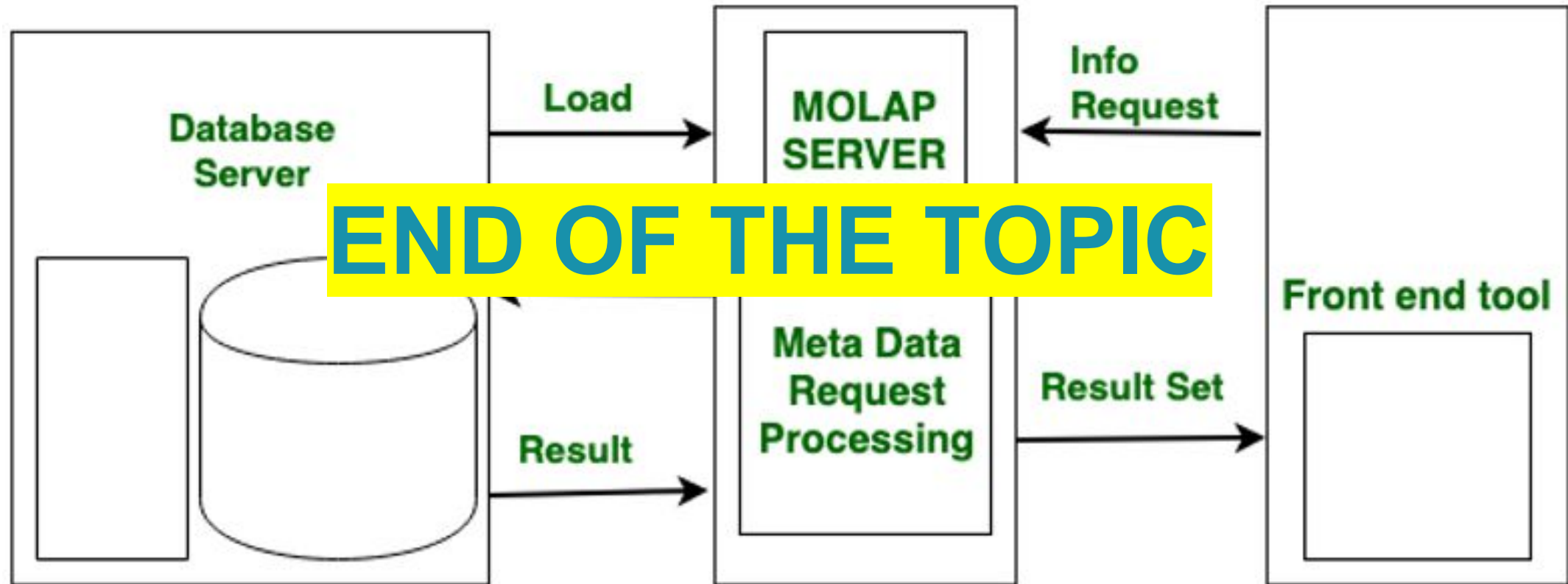
When using an OLAP function, a window is specified that defines the rows over which the function is applied, and in what order. The set of rows is defined, relative to the current row, as either a range or a number of rows preceding and following. For example, you can calculate an average over the previous three-month period. The following OLAP functions are available:

Window aggregate functions - these return a value for a specified set of rows in the input. Some of the commonly used window aggregate functions are listed below:

- **AVG - Computes the average, for a set of rows, of a numeric expression or set of unique values.**
- **MAX - Returns the maximum expression value found in each group of rows.**
- **MIN - Returns the minimum expression value found in each group of rows.**
- **SUM - Returns the total of the specified expression for each group of rows.**

Window ranking functions - these return the rank of a row relative to the other rows in a partition. The supported window ranking functions are listed below:

- **CUME_DIST** - Computes the position of one row returned from a query with respect to the other rows returned by the query, as defined by the ORDER BY clause.
- **DENSE_RANK** - Calculates the rank of a value in a group. The difference between RANK and DENSE_RANK is that RANK leaves a gap in the ranking sequence when there are ties between values.
- **PERCENT_RANK** - Calculates a percentage for the rank rather than a fractional amount. This function returns a decimal value between 0 and 1.
- **RANK** - Computes the rank of each row returned from a query with respect to the other rows returned by the query, as defined by the ORDER BY clause.
- **Row numbering functions** - this uniquely number the rows in a partition.
- **ROW_NUMBER** - Assigns a unique number to each row, which is determined by the ORDER BY clause.



END OF THE TOPIC